

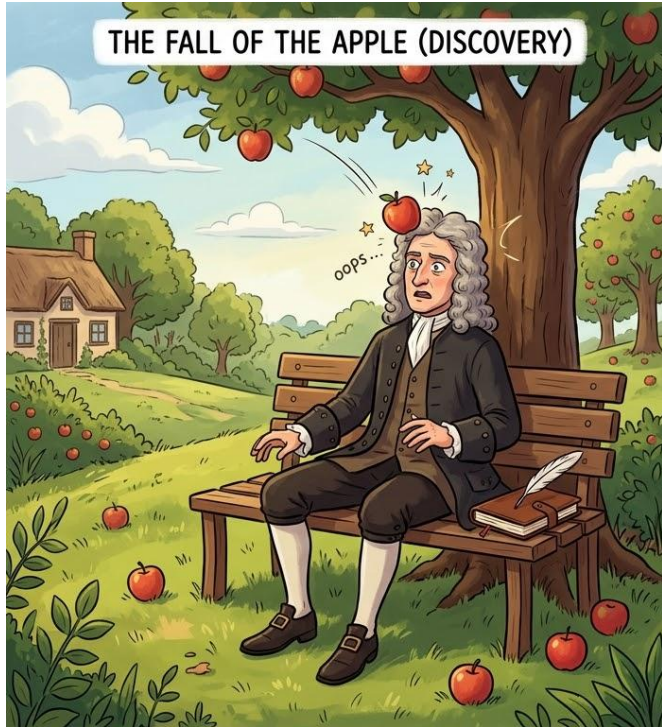
CS3213: Foundations of Software Engineering

“The Scientific Method” to Debugging

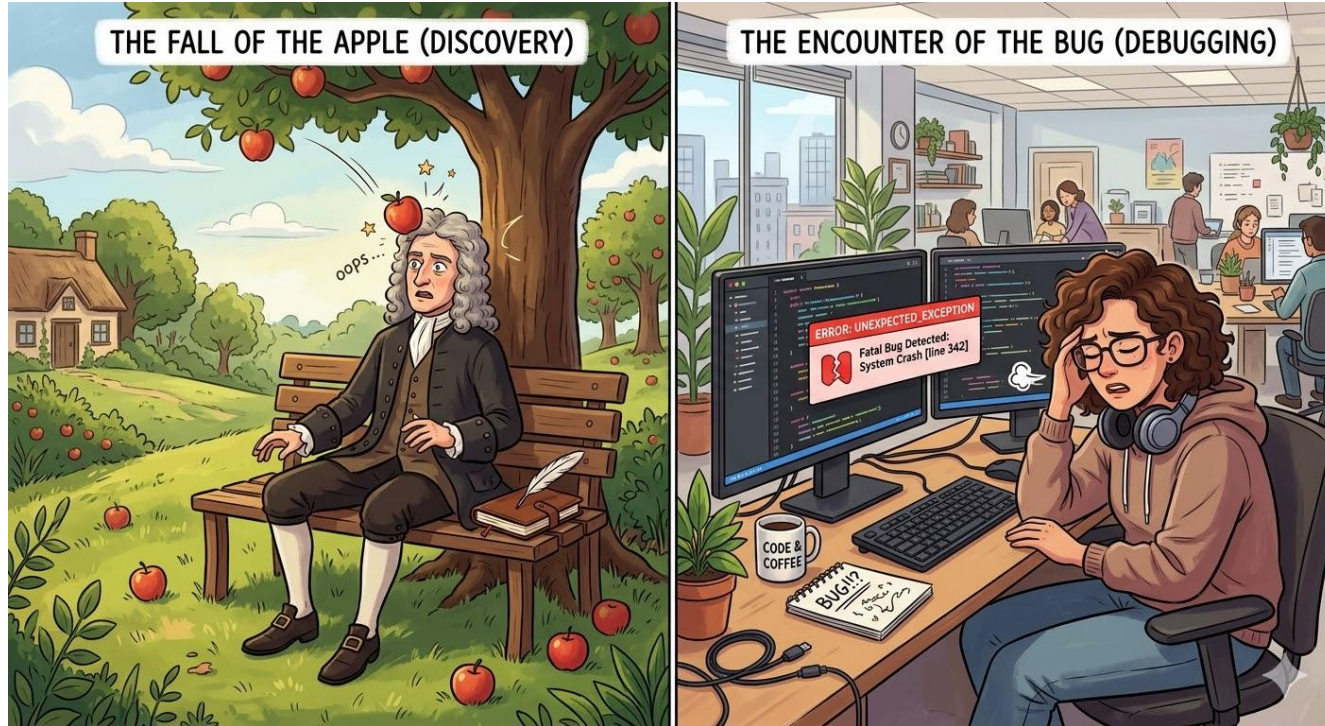
“The Scientific Method” to Debugging



Isaac Newton and Gravity



Isaac Newton and Gravity





“The Scientific Method” to Debugging

1. Formulate a question



“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior



“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.



“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.
4. **Test the prediction** (and thus the hypothesis) in an experiment. If the prediction holds, confidence in the hypothesis increases; otherwise, it decreases.



“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.
4. **Test the prediction** (and thus the hypothesis) in an experiment. If the prediction holds, confidence in the hypothesis increases; otherwise, it decreases.
5. Repeat Steps 2–4 until there are no discrepancies between hypothesis and predictions and/or observations.

Example: removeHtmlMarkup

```
public static String removeHtmlMarkup(String html) {
```



```
"Here's some <strong>strong argument</strong>."
```

```
return ...;
```



```
"Here's some strong argument."
```

```
}
```



Question

Why does the method return the correct result for some, but not for other inputs?

Finding a Hypothesis

Input	Expected Output	Output	Outcome
<code>foo</code>	foo	foo	✓
<code>"foo"</code>	"foo"	foo	✗
<code>"foo"</code>	"foo"	<code>foo</code>	✗
<code><b id="bar">foo</code>	foo	foo	✓

Finding a Hypothesis

Input	Expected Output	Output	Outcome
<code>foo</code>	foo	foo	✓
<code>"foo"</code>	"foo"	foo	✗
<code>"foo"</code>	"foo"	<code>foo</code>	✗
<code><b id="bar">foo</code>	foo	foo	✓

Hypothesis 1: The tag `<>` is always stripped from the input. ✗

Finding a Hypothesis

Input	Expected Output	Output	Outcome
<code>foo</code>	foo	foo	✓
<code>"foo"</code>	"foo"	foo	✗
<code>"foo"</code>	"foo"	<code>foo</code>	✗
<code><b id="bar">foo</code>	foo	foo	✓

Hypothesis 2: Double quotes (") are stripped from the tagged input. ✓

Finding a Hypothesis

Input	Expected Output	Output	Outcome
<code>foo</code>	foo	foo	✓
<code>"foo"</code>	"foo"	foo	✗
<code>"foo"</code>	"foo"	<code>foo</code>	✗
<code><b id="bar">foo</code>	foo	foo	✓

Hypothesis 3: Tags in double quotes are not stripped. ✓

Testing a Hypothesis

- Focus: Double quotes are stripped from the tagged input.

`“foo”`
↓
`public static String removeHtmlMarkup(String html)` → `foo`

We can try more inputs to test the hypothesis!

Finding a Hypothesis

Input	Expected Output	Output	Outcome
foo	foo	foo	✓
"foo"	"foo"	foo	✗
"foo"	"foo"	foo	✗
<b id="bar">foo	foo	foo	✓
"foo"	"foo"	foo	✗

Hypothesis 2: Double quotes (") are stripped from the tagged input.

Refining the Hypothesis

“foo”



```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray()) {  
        assert !tag;  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>' && !quote) {  
            tag = false;  
        } else if (c == '"' || c == '\\' && tag) {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

Refined hypothesis: The error is due to tag being set. We can test the refined hypothesis using an ***assert*** statement

Printf Debugging

“foo”



```
public static String removeHtmlMarkup(String html) {  
    boolean tag = false;  
    boolean quote = false;  
    String out = "";  
    for (char c : html.toCharArray())  
        assert !tag;  
        if (c == '<' && !quote) {  
            tag = true;  
        } else if (c == '>' && !quote) {  
            tag = false;  
        } else if (c == '"' || c == '\\' && tag) {  
            quote = !quote;  
        } else if (!tag) {  
            out += c;  
        }  
    }  
    return out;  
}
```

Our expectation was that the execution would fail. Since it did not, we reject our hypothesis

~~The error is due to tag being set.~~

Refining the Hypothesis

- ~~The error is due to tag being set.~~
 - We now know that the tag is not set
- Hypothesis: the error is due to the quote condition evaluating to true
 - The quote should not be set if we are not in a tag!

```
else if (c == '"' || c == '\\') && tag) {  
    quote = !quote;  
}
```

Refining the Hypothesis

```
else if (c == '"' || c == '\\' && tag) {  
    assert false;  
    quote = !quote;  
}
```

Failure Trace

java.lang.AssertionError

at HTMLUtils.removeHtmlMarkup(HTMLUtils.java:49)

at TestHTMLUtils.testRemoveTextFoo(TestHTMLUtils.java:24)

at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)

at java.base/java.util.ArrayList.forEach(ArrayList.java:1511)

The assertion fails, so the condition
must be the culprit!

Confirmed Hypothesis

- ~~The error is due to tag being set.~~
 - We now know that the tag is not set
- The error is due to the quote condition evaluating to true ✓
- The logic is the same for single and double quotes. Can we generalize the hypothesis?
 - Double **and single quotes** are stripped from the tagged input.

Finding a Hypothesis

Input	Expected Output	Output	Outcome
foo	foo	foo	✓
"foo"	"foo"	foo	✗
"foo"	"foo"	foo	✗
<b id="bar">foo	foo	Foo	✓
"foo"	"foo"	foo	✗
'foo'	'foo'	'foo'	✓

Surprisingly, the logic works for
single quotes

Fixing the Bug

- In most languages, the “and” operator takes precedence over “or”, which means that we require parentheses
- Hypothesis: the error is due to the quote condition evaluating to True
 - Double check with the previous input/output samples

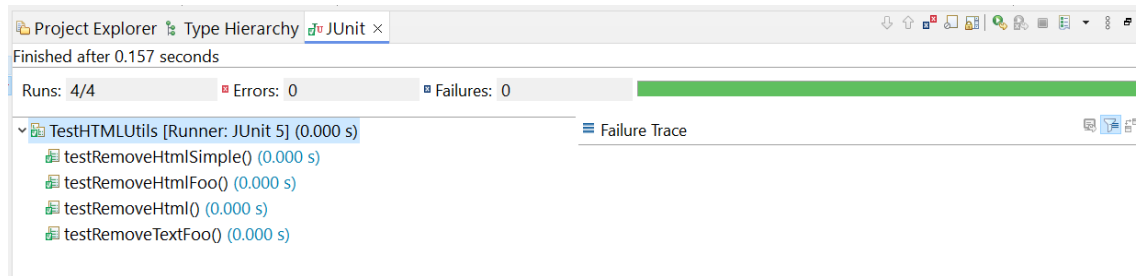
```
else if (c == '"' || c == '\\' && tag) {  
    quote = !quote;  
}
```

Diagnosis

- In terms of scientific process, we now have a *theory*, which is a hypothesis that
 - Is consistent with earlier observations
 - Predicts future observations (i.e., the program's behavior)
- In the context of debugging, we typically speak of a *diagnosis*

When you do have a diagnosis that explains **both causality** (how the failure came to be), and **incorrectness** (how to correct the code accordingly), then (and only then!) is it time to actually fix the code accordingly

Fix



```
else if ((c == '"' || c == '\\') && tag) {  
    quote = !quote;  
}
```

Our diagnosis was correct!



Debugging Process

- For this simple example, directly using a debugger (or printf debugging) might have also allowed us to find the bug
- For complex issues, which might have days or weeks to debug, having such a systematic debugging strategy is useful
 - In such cases, it also helps to systematically log and create a table as we did

Rubberducking

- A useful way to revisit your observations and come up with hypothesis is to explain the problem to someone else
- **Rubberducking**: since the “someone else” can be passive, using a rubber duck or another inanimate object can be used to explain the issue





“The Scientific Method” to Debugging

1. Formulate a question
2. Come up with an ***hypothesis*** based on the knowledge obtained while formulating the question that may explain the observed behavior
3. Determining the logical consequences of the hypothesis, **formulate a prediction that can support or refute the hypothesis**. Ideally, the prediction would distinguish the hypothesis from likely alternatives.
4. **Test the prediction** (and thus the hypothesis) in an experiment. If the prediction holds, confidence in the hypothesis increases; otherwise, it decreases.
5. Repeat Steps 2–4 until there are no discrepancies between hypothesis and predictions and/or observations.